GhostLock (CVE-2026-43499) — 去址化 **Before/After** 比較

> 用途：作為寄給 Rokid（`degang.xu@rokid.com`）的「redacted technical comparison」。
>
> 紅線處理：本文件已移除所有絕對 **kernel** 位址、裝置序號與武器化細節（heap grooming、reclaim、controlled-write 參數、完整 exploit）。指令以函式相對偏移表示， 呼叫/跳轉目標改以符號名標示。完整的原檔反組譯（含絕對位址）僅保留於私人工作區，不外寄。
>
> 版本識別與雜湊見 §1；修補判定依據見 §2 與 §3.2。

---

1. 版本與映像識別

1.1 受影響版

項目	值
firmware/build	`1.22.009-20260710-150201`
OTA 套件檔名	`RVE01-ota-1.22.009-20260710-150201.zip`
完整套件雜湊	SHA-256 163ab2058d8e8789f82e962c7bfccbe6ca9725437d15adaf4bc4a811bafc94e5（MD5 未留存；2026-09-16 由原廠 CDN 重新下載並實測，與我方留存清單一致）
Android 版本	12（`SKQ1.240613.001`、`user/release-keys`）
kernel 版本字串	`5.10.209-perf-gd908d71a0404-ab1.22.009-20260710-150201`
Android SPL	`2024-07-05`

1.2 修補版

項目	值
firmware/build	`1.25.012-20260901-150201`
OTA 套件檔名	`RVE01-ota-1.25.012-20260901-150201.zip`（約 1 GB）
完整套件雜湊	MD5 bec406d9e28ee779d8e876e57ba69f51（與原廠 OTA metadata 所載一致；2026-09-16 由原廠 CDN 重新下載並實測） SHA-256 a24412ad8bc19d95e605952a3bf32e91e3692002e5195139650f73056e8985f5
Android 版本	12（`SKQ1.240613.001`、`user/release-keys`）
kernel 版本字串	`5.10.209-perf-g9a43350cc9ee-ab1.25.012-20260901-150201`
Android SPL	`2024-07-05`

> 注意：兩版的 `post-security-patch-level` 都是 `2024-07-05`，因此單看 **SPL** 會誤判為「未修補」。 判斷修補與否必須反組譯 `remove_waiter()`。

---

2. 判斷基準（**upstream patch**）

- CVE：`CVE-2026-43499`（"GhostLock"），`kernel/locking/rtmutex.c` 的 `remove_waiter()` use-after-free。
- 官方修補 commit：`3bfdc63` — "*rtmutex: Use waiter::task instead of current in remove_waiter()*"。
- 修補語意：把 `remove_waiter()` 內操作 `current` 的動作，改為操作 `waiter->task`。
- ARM64 判別特徵（須看 **`x20`** 的來源，而非只搜尋 **`#0x30`** 載入）：
- `remove_waiter()+0x040` 為 `mrs x20, sp_el0`（`x20 = current`）→ 漏洞版。

- `remove_waiter()+0x040` 為 `ldr x20, [x23, #0x30]`（`x23 = waiter`，`x20 = waiter->task`）→ 修補版。
- ⚠ 僅憑「函式內是否出現 **`ldr …, #0x30`**」會誤判：1.22.009 的 `remove_waiter()` 內 亦有一處 `ldr x14, [x9, #0x30]`（`+0x160`），但其基底是 `x9`、與 `waiter->task` 無關。 必須確認 `+0x044`（`pi_lock`）、`+0x088`（`pi_blocked_on`）與 `+0x210`（`adjust_prio_chain` 的 task 參數）三者是否都由 **`+0x040`** 載入的 **`x20`** 餵入。

---

3. `remove_waiter()` 控制流程對照（去址化）

> 本節清單由 §1.1 / §1.2 所列 OTA 套件解出 `boot.img` → kernel Image 後反組譯產生； 函式位置以符號 `remove_waiter` 定位，指令位址一律換算為函式相對偏移。 兩版使用同一套工具鏈（`vmlinux-to-elf` 還原符號 + Capstone 反組譯）處理， 因此兩份清單可直接對照。
>
> 逐位元組比對結果：兩版的 `remove_waiter()` 大小完全相同（**`0x28c` = 163** 條指令）， 且機器碼只有一條 **4-byte** 指令不同——即 `+0x040`。其餘 162 條指令（含所有呼叫目標）完全相同。

3.1 受影響版 **1.22.009** —— 漏洞版（**`current`**）

以下為 1.22.009 `remove_waiter()` 的去址化清單（絕對位址已換算為函式相對偏移，呼叫目標以符號表示）：

```
remove_waiter:
  +0x000  paciasp
  +0x004  str     x30, [x18], #8
  +0x008  stp     x29, x30, [sp, #-0x40]!
  ...
  +0x01c  ldr     x22, [x0, #0x10]
  +0x020  mov     x19, x0                   ; x19 = lock
  +0x024  mov     x23, x1                   ; x23 = waiter
  ...
  +0x040  mrs     x20, sp_el0               ; x20 = current            ← 漏洞版關鍵
  +0x044  add     x21, x20, #0x85c          ; x21 = &current->pi_lock
  +0x048  mov     x0, x21
  +0x04c  bl      _raw_spin_lock            ; raw_spin_lock(&current->pi_lock)
  ...
  +0x088  str     xzr, [x20, #0x888]        ; current->pi_blocked_on = NULL  ← 清錯對象
  +0x08c  bl      _raw_spin_unlock
  ...
  +0x160  ldr     x14, [x9, #0x30]          ; 註：基底為 x9，與 waiter->task 無關
  ...
  +0x210  mov     x5, x20                   ; 第 5 參數 = current
  +0x214  bl      rt_mutex_adjust_prio_chain
  ...
 （函式內無 `mrs` 以外的 current 替代來源；`current` 貫穿 pi_lock / pi_blocked_on /
   adjust_prio_chain 三個用途）
```

判讀：`x20` 直接由 `mrs x20, sp_el0`（`current`）取得，`pi_lock`（`#0x85c`）、 `pi_blocked_on = NULL`（`#0x888`）與 `rt_mutex_adjust_prio_chain` 的 task 參數全部以 **`current`** 為目標。 當 `current`（requeue 端）不等於 `waiter->task`（被 requeue 的 waiter 所屬 task）時， `current->pi_blocked_on` 會被錯誤清除 —— 此即 CVE-2026-43499 的 UAF 前提。

3.2 修補版 **1.25.012** —— 已修補（**`waiter->task`**）

以下摘自 1.25.012 `remove_waiter()`，已將絕對位址轉為函式相對偏移，呼叫目標以符號表示：

```
remove_waiter:
  +0x000  paciasp
  +0x004  str     x30, [x18], #8
```

```
  +0x008  stp     x29, x30, [sp, #-0x40]!
  ...
  +0x040  ldr     x20, [x23, #0x30]        ; x20 = waiter->task（rt_mutex_waiter.task）
  +0x044  add     x21, x20, #0x85c         ; x21 = &task->pi_lock
  +0x04c  bl      raw_spin_lock             ; raw_spin_lock(&task->pi_lock)
  ...
  +0x088  str     xzr, [x20, #0x888]       ; task->pi_blocked_on = NULL  ← 正確目標
  ...
  +0x210  mov     x5, x20                   ; 第 5 參數 = task
  +0x214  bl      rt_mutex_adjust_prio_chain
  ...
（全程無 mrs ... sp_el0 → 不使用 current）
```

判讀：`x20` 由 `waiter->task`（`+0x040`，`[x23, #0x30]`）載入後，`pi_lock`（`#0x85c`）、 `pi_blocked_on = NULL`（`#0x888`）與 `rt_mutex_adjust_prio_chain` 的 task 參數全部以 **`waiter->task`** 為目標，不再取用 `current`（函式內無 `mrs …, sp_el0`）。此即 upstream commit `3bfdc63` 的語意。

> 兩版對照的關鍵事實：`remove_waiter()` 在兩版中大小相同（**`0x28c`**）， 機器碼逐位元組比對只有 **`+0x040`** 這一條指令不同（`mrs x20, sp_el0` → `ldr x20, [x23, #0x30]`），其餘 162 條指令完全一致。 這是一次最小化的 backport：只改一個 task 來源，即消除 CVE-2026-43499 的前提。

---

4. 關鍵差異結論

特徵	**1.22.009**	**1.25.012**
`+0x040` 的 `x20` 來源	`mrs x20, sp_el0`（current）	`ldr x20, [x23, #0x30]`（waiter->task）
讀取 `current`（`mrs …, sp_el0`）	是（漏洞版）	否
`+0x044` `pi_lock` 對象	`current->pi_lock`	`waiter->task->pi_lock`
`+0x088` `pi_blocked_on = NULL` 寫入目標	`current->pi_blocked_on`	`waiter->task->pi_blocked_on`
`+0x210` `adjust_prio_chain` 的 task 參數	`current`	`waiter->task`
其他 `#0x30` 載入	有，但基底為 `x9`（`+0x160`），與 `waiter->task` 無關	—
函式大小	`0x28c`（163 條指令）	`0x28c`（163 條指令，相同）
機器碼差異	僅 `+0x040` 一條 4-byte 指令	（同左）
kernel git hash（版本字串）	`gd908d71a0404`	`g9a43350cc9ee`（不同建置）
upstream `3bfdc63` 語意	未含	含
實機 CVE-2026-43499 exploit	成功（`uid 2000 → uid 0`）	前置條件不再成立

---

5. 實機 **before/after** 證據（去識別）

- **Before**（**1.22.009**）：ADB shell 由低權限 `shell`（`uid=2000`、`u:r:shell:s0`、SELinux Enforcing）取得 `uid=0`、`u:r:kernel:s0`、SELinux Permissive、完整 effective capabilities。
- **After**（**1.25.012**）：`remove_waiter()` 已改用 `waiter->task`，`current` ≠ `waiter->task` 的 PI 場景下不再寫入錯誤 task 的 `pi_blocked_on`，漏洞前置條件被移除。

> 不在本文件提供：完整 log、kernel 位址、序列號、ADB keys、heap 參數與 exploit 原始碼。
